重庆大学



课 程 设 计 报 告

姓　名	
学　号	

课程名称 软件综合实践

设计题目 _____

课程教师 _____

设计起止日期 2021.7.5-2021.7.23

评　　分 _____

大数据与软件学院

目 录

1 绪论 4
1.1 项目背景 4
1.2 项目目标与意义 4
1.3 主要技术路线 4
2 需求分析 4
2.1 角色与总体需求 4
2.2 功能需求 5
2.3 非功能需求 5
3 数据库设计 6
3.1 数据流分析 6
3.2 概念结构设计 6
3.3 逻辑结构与数据字典 6
3.4 物理设计与建库策略 7
4 系统设计 7
4.1 总体架构 7
4.2 核心类设计 8
4.3 文章发布时序设计 8
4.4 主要活动流程 8
4.5 接口设计 8
5 编码实现 9
5.1 开发与运行环境 9
5.2 一键启动实现 9
5.3 用户认证实现 9
5.4 文章智能处理实现 10

5.5 图片与音乐实现 10
5.6 主要界面 10
6 程序调试情况 10
6.1 测试方法与结果 10
6.2 典型问题与分析 11
6.3 关键代码边界检查 11
7 系统总结 11
7.1 系统特点 11
7.2 存在的问题 12
7.3 改进方向 12
8 课程设计总结 12
9 参考文献 12

1 绪论

1.1 项目背景

随着个人创作内容从单一文字逐步扩展到图片、音乐与互动信息，传统静态博客难以同时满足内容管理、个性化展示和多用户公开访问的需要。本项目设计并实现一个前后端分离的个人博客系统 BlogSystem：博主可以管理文章、相册、音乐和友情链接，访客可以通过公开地址浏览内容、留言并参与文章评论。

系统以“易启动、易扩展、界面沉浸”为主要目标。项目根目录提供 start.sh 与 stop.sh，用于一次性完成依赖检查、前后端启动、日志管理和进程关闭；后端启动时自动初始化 SQLite 数据库，前端以响应式页面组织各功能模块。

1.2 项目目标与意义

- 完成注册登录、Bearer 会话鉴权与多用户数据隔离。
- 实现文章全生命周期管理，支持 Markdown、摘要生成、关键词匹配和自动标签。
- 实现相册上传、裁剪、分组浏览，以及音乐检索、播放、歌词与播放列表管理。
- 提供公开主页、公开文章、相册、友链、留言和评论回复功能。
- 通过前后端分离和清晰的模型/路由/服务分层，形成可维护、可继续扩展的课程实践项目。

1.3 主要技术路线

层次	采用技术	作用
前端	Next.js 16.2.9、React 19.2.4、TypeScript、Tailwind CSS 4	路由、组件化视图、交互状态与响应式样式
后端	FastAPI 0.138.2、SQLModel 0.0.39、Uvicorn	REST API、参数校验、业务编排与数据访问
数据	SQLite、SQLAlchemy	关系数据持久化、索引和事务
工程	uv、pnpm、Shell 启停脚本	依赖锁定、开发启动与日志管理

2 需求分析

2.1 角色与总体需求

系统包含博主和访客两类主要角色。博主是内容所有者，需要完成账号注册登录、内容发布与维护、相册管理、个性化设置和评论治理；访客无需登录即可访问指定用户名的公开

博客，查看内容并发布留言或评论。系统必须保证所有受保护数据均按 owner_id 过滤，避免不同博主之间的数据串读和越权修改。

图 2-1 系统功能模块图

2.2 功能需求

编号	功能模块	主要需求	优先级
F01	用户认证	注册、登录、当前用户查询、会话令牌校验	高
F02	文章管理	发布、更新、删除、详情、搜索、标签筛选、Markdown 展示	高
F03	相册管理	多文件选择、裁剪、上传、相册分组、灯箱浏览、删除	高
F04	公开博客	按 username 读取文章、照片、友链和留言	高
F05	评论留言	访客评论、二级回复、博主删除；访客留言与列表展示	中
F06	音乐中心	歌曲搜索、播放地址、歌词、播放列表和循环模式	中
F07	个性化	头像、简介、壁纸、夜间模式和玻璃透明度	中
F08	工程运行	一键启动、健康检查、日志保存和安全关闭	高

图 2-2 系统用例图

2.3 非功能需求

- 安全性：密码使用带随机盐的 PBKDF2-SHA256 进行 210000 次迭代，敏感写操作必须校验 Bearer Token 和资源归属。
- 可用性：主要页面提供加载、空状态和错误提示；start.sh 自动发现 pnpm/corepack/npm 并创建运行日志目录。
- 性能：列表查询按创建时间倒序并在数据库层过滤；前端静态页面可预渲染，动态公开页面按请求服务端渲染。
- 兼容性：前端采用响应式布局，后端通过 CORS 配置允许本地常用端口访问。

- 可维护性：前端按 app/components/lib 分层，后端按 api/models/services/db/core 分层。

3 数据库设计

3.1 数据流分析

客户端请求首先到达 FastAPI 路由。受保护接口从 Authorization 请求头提取会话令牌，查询 User 表确认身份；业务层随后执行摘要、标签、文件或音乐数据处理，并通过 SQLModel 会话完成事务。公开路由以 username 定位博客所有者，再用 owner_id 对内容表进行过滤。

图 3-1 系统数据流图

3.2 概念结构设计

核心实体为 User、Post、Photo、FriendLink、Message 和 Comment。User 与各内容实体均为一对多关系；Post 与 Comment 为一对多关系；Comment 通过 parent_id 形成自关联，以支持一层回复结构。

图 3-2 数据库 E-R 图

3.3 逻辑结构与数据字典

表名	关键字段	约束 / 索引	说明
user	id, email, username, password_hash, session_token, created_at	email/username/session_token 唯一并索引	用户身份与登录会话
post	id, owner_id, title, content, summary, tags, keyword_matches	owner_id 外键并索引	文章正文及自动处理结果
photo	id, owner_id, title, image_url, description	owner_id 外键并索引	照片文件名及相册编码
friendlink	id, owner_id, name, url, description	owner_id 外键并索引	友情链接
message	id, owner_id, author, content, created_at	owner_id 外键并索引	公开留言板
comment	id, post_id, owner_id, parent_id, author, content	post_id/owner_id 索引	文章评论和回复

3.4 物理设计与建库策略

默认数据库地址为 sqlite:///./blogsystem.db。应用 lifespan 启动阶段调用 init_db()，先执行 SQLModel.metadata.create_all，再检查旧数据库是否缺少 owner_id、cover_url、parent_id 或 username 字段，并通过 ALTER TABLE 进行兼容迁移。该策略适合课程设计和本地演示；生产环境应改用 Alembic 等版本化迁移工具。

```
CREATE TABLE post (
  id INTEGER PRIMARY KEY,
  owner_id INTEGER REFERENCES user(id),
  title VARCHAR(160) NOT NULL,
  content VARCHAR NOT NULL,
  category VARCHAR(60) NOT NULL,
  summary VARCHAR(280) NOT NULL,
  tags JSON,
  keyword_matches JSON,
  created_at DATETIME NOT NULL,
  updated_at DATETIME NOT NULL
);

SELECT * FROM post
WHERE owner_id = :current_user_id
  AND (title LIKE :pattern OR content LIKE :pattern OR summary LIKE :pattern)
ORDER BY created_at DESC;
```

4 系统设计

4.1 总体架构

系统采用浏览器/服务器（B/S）架构和前后端分离模式。Next.js 负责页面路由、组件状态和视觉交互；FastAPI 提供 JSON API；SQLModel 同时承担数据模型、参数校验和 ORM 映射；SQLite 保存结构化数据，uploads/photos 保存照片文件，音乐服务通过 requests 调用第三方接口。

图 4-1 系统总体架构图

4.2 核心类设计

图 4-2 核心领域类图

4.3 文章发布时序设计

图 4-3 文章发布时序图

4.4 主要活动流程

图 4-4 用户访问与内容操作活动图

4.5 接口设计

方法	路径	鉴权	功能
POST	/api/auth/register	否	注册并返回 token 与用户信息
POST	/api/auth/login	否	登录并轮换会话 token
GET/POST	/api/posts	是	文章列表、搜索与发布
GET/PUT/DELETE	/api/posts/{id}	是	文章详情、编辑和删除
GET/POST	/api/photos	是	照片列表与 multipart 上传
GET	/api/public/{username}/posts	否	公开文章列表
GET/POST	/api/public/posts/{id}/comments	否	评论列表与创建
GET	/api/music/search|song|lyric	否	音乐搜索、播放地址与歌词

5 编码实现

5.1 开发与运行环境

项目	要求 / 实测环境
操作系统	macOS（亦可在支持 Python/Node.js 的平台运行）
Python	项目声明 ≥3.12；本次依赖环境使用 CPython 3.14.6
Node.js / 包管理	Node.js + pnpm 11.9.0
后端端口	http://127.0.0.1:8000，Swagger 文档 /docs
前端端口	http://localhost:3000
启动方式	项目根目录执行 ./start.sh；结束执行 ./stop.sh
日志位置	.blogsystem-run/logs/backend.log 与 frontend.log

5.2 一键启动实现

根目录 start.sh 先检查 uv，再按 pnpm、corepack pnpm、npm 的顺序选择前端运行器；缺少 backend/.env 时会从 .env.example 复制；缺少虚拟环境或 node_modules 时自动安装依赖；最后以后台进程启动 Uvicorn 和 Next.js，并将 PID 与日志写入 .blogsystem-run。

```
cd "/Users/skyler/Python Project/BlogSystem"
./start.sh

# 服务地址
# 前端：http://localhost:3000
# 后端：http://127.0.0.1:8000
# 接口文档：http://127.0.0.1:8000/docs

./stop.sh
```

5.3 用户认证实现

注册接口先规范化邮箱和用户名，并处理用户名冲突；密码使用 secrets.token_hex(16) 生成随机盐，调用 hashlib.pbkdf2_hmac 计算摘要。登录时以 hmac.compare_digest 做恒定时间比较，成功后生成新的随机会话令牌。受保护路由统一依赖 get_current_user。

```
digest = hashlib.pbkdf2_hmac(
    "sha256", password.encode("utf-8"),
    salt.encode("utf-8"), 210_000,
).hex()
```

```
token = authorization.removeprefix("Bearer ").strip()
user = session.exec(select(User).where(User.session_token == token)).first()
```

5.4 文章智能处理实现

发布或更新文章时，服务层从标题和正文中提取关键词，去除 Markdown 标记并生成不超过 280 字的摘要；generate_tags 根据技术规则、用户关键词命中和词频生成标签。处理结果与正文同时保存，前端据此显示标签并支持搜索。

5.5 图片与音乐实现

照片页使用 react-easy-crop 提供自由、1:1、4:3、16:9、3:4 和 9:16 裁剪比例；前端将裁剪结果转为 File 后，以 multipart/form-data 上传。后端用 UUID 生成文件名，避免重名覆盖。音乐模块维护全局 HTMLAudioElement、播放列表、音量、循环模式与歌词解析状态，并并行请求歌曲详情和歌词。

5.6 主要界面

以下截图均在项目根目录执行 start.sh 后，通过 Safari 访问实际运行系统获得。截图使用演示账号和测试数据，展示登录、博主主页、文章管理与相册管理的真实界面。



图 5-1 系统实际登录界面



图 5-2 系统实际博主主页



图 5-3 系统实际文章管理界面



图 5-4 系统实际相册管理界面

6 程序调试情况

6.1 测试方法与结果

检查项	执行方式	结果
后端启动	./start.sh 后读取 backend.log	Uvicorn 启动完成，数据库初始化成功
健康检查	GET /api/health	200，返回 {status: ok}
注册与鉴权	POST /api/auth/register	201，返回会话令牌和用户对象
文章发布	带 Bearer Token POST /api/posts	201，自动生成摘要、标签与匹配

检查项	执行方式	结果
		词
公开读取	GET /api/public/reportdemo/posts	200，实测 total=2
后端语法	python -m compileall backend/app	通过
前端生产构建	pnpm build	通过；静态与动态路由均成功生成
ESLint	pnpm lint	未通过：6 个错误、3 个警告，集中于 effect 内同步 setState 和未使用符号

6.2 典型问题与分析

问题一：早期数据库没有 owner_id、cover_url、parent_id 或 username 字段，新代码直接查询会失败。解决方式是在 init_db 后检查 SQLite 表结构并执行增量 ALTER TABLE，同时回填已有用户的 username。长期改进应引入正式迁移工具。

问题二：公开页面与博主管理页面的数据来源不同。管理页面需要 Bearer Token，公开页面按 username 过滤；若错误复用受保护接口，访客将收到 401。项目通过独立 public_routes.py 明确分离两类访问路径。

问题三：图片数据库字段的历史长度与模型声明可能存在差异，且文件扩展名目前直接取自上传文件名。建议增加 MIME 类型白名单、文件大小限制和安全扩展名映射。

问题四：当前 ESLint 的 react-hooks/set-state-in-effect 规则报告 6 处错误，虽然 pnpm build 与 TypeScript 检查通过，但持续集成若以 lint 为门禁将失败。应将同步重置移动到事件、派生状态或异步回调中，并清理未使用的变量和组件。

6.3 关键代码边界检查

- 登录失败统一返回 401，不区分邮箱不存在或密码错误。
- 删除友链、文章、照片和评论均校验资源归属。
- 评论回复创建前检查父评论存在且属于同一文章。
- 音乐第三方请求设置 15 秒超时，并将上游异常映射为 502。
- 照片文件读取前检查 os.path.isfile，缺失资源返回 404。

7 系统总结

7.1 系统特点

- 完整的前后端分离实现，覆盖认证、内容、媒体、互动和公开展示。
- 面向多用户的数据归属设计，公开路由与管理路由边界清晰。

- 文章摘要、标签、关键词匹配和 Markdown 展示提升内容组织能力。
- 相册裁剪、沉浸式音乐、壁纸与玻璃透明度增强视觉体验。
- 根目录启停脚本降低环境准备和课堂演示成本。

7.2 存在的问题

- 会话令牌长期保存在数据库和浏览器 localStorage，未设置过期时间、刷新令牌或主动吊销列表。
- SQLite 和本地文件存储适合单机演示，但并发、备份与水平扩展能力有限。
- 数据迁移依赖手写 ALTER TABLE，缺少可追踪的迁移版本。
- 第三方音乐接口稳定性和版权可用性不完全受系统控制。
- 自动化测试不足，当前 ESLint 仍存在阻断级问题。

7.3 改进方向

1. 引入 JWT 过期策略、HttpOnly Cookie、CSRF 防护、登录限流和密码重置流程。
2. 使用 PostgreSQL、Alembic 和对象存储，完善外键级联、备份和迁移。
3. 增加 pytest API 测试、Playwright 端到端测试与 GitHub Actions 持续集成。
4. 修复 lint 问题，完善可访问性、移动端断点和图片懒加载指标。
5. 增加文章草稿、分页、全文检索、管理员审核与数据导出功能。

8 课程设计总结

本次课程设计将需求分析、数据库建模、接口设计、前端交互和工程运行串联为一个完整过程。实践中最重要的认识是：一个可用系统不仅需要页面和接口“能运行”，还需要清晰的数据归属、异常边界、可复现的启动方式以及可验证的构建结果。

在后端部分，通过 FastAPI 的依赖注入、SQLModel 的声明式模型和事务会话，掌握了从请求校验到数据持久化的完整链路；在前端部分，通过 Next.js App Router、React Hook、全局音频状态和响应式样式，理解了页面路由与复杂交互状态的组织方式。数据库兼容迁移、公开/私有接口分流和文件上传处理，则进一步体现了真实项目中“边界条件”往往比理想流程更需要关注。

调试结果也说明“可以构建”不等于“质量完全达标”。生产构建通过，但 ESLint 暴露了 effect 内同步状态更新和未使用代码等问题。后续应把静态检查、单元测试、接口测试和端到端测试纳入持续集成，使系统从课程演示级逐步提升到可长期维护的应用。

9 参考文献

[1] FastAPI. FastAPI Documentation [EB/OL]. https://fastapi.tiangolo.com/.

[2] Vercel. Next.js Documentation [EB/OL]. https://nextjs.org/docs.

[3] Meta. React Documentation [EB/OL]. https://react.dev/.

[4] Sebastián Ramírez. SQLModel Documentation [EB/OL]. https://sqlmodel.tiangolo.com/.

[5] SQLAlchemy Authors. SQLAlchemy Documentation [EB/OL]. https://docs.sqlalchemy.org/.

[6] SQLite Consortium. SQLite Documentation [EB/OL]. https://www.sqlite.org/docs.html.

[7] OWASP Foundation. Password Storage Cheat Sheet [EB/OL]. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

[8] Tailwind Labs. Tailwind CSS Documentation [EB/OL]. https://tailwindcss.com/docs.